AI-HOS: Intelligent Hospital Operating System

Comprehensive System Documentation & Architecture Report

Version: 1.0 | **Date:** March 8, 2026 **Platform:** Next.js 15 | Supabase | n8n Workflow Automation **Author:** Narravula Ram Charan Teja **Organization:** Advera Healthcare Technologies

---

Table of Contents

1. Executive Summary
2. System Architecture
3. Technology Stack
4. Authentication & Security
5. Multi-Tenancy Architecture
6. User Roles & Access Control
7. Feature Modules
8. User Flows & Journeys
9. Database Schema
10. API Architecture
11. Real-Time Engine
12. Workflow Automation (n8n)
13. Design System & UX
14. Print & Document Generation
15. Deployment & Infrastructure
16. Security Audit Summary
17. Performance Optimizations
18. Known Issues & Limitations
19. Complexity Analysis
20. Future Roadmap

---

1. Executive Summary

AI-HOS is a production-grade, multi-tenant Hospital Operating System that digitizes the entire outpatient and inpatient workflow — from patient registration and appointment booking through consultation, prescription, pharmacy dispensing, lab testing, billing, and discharge.

Key Differentiators

- **Multi-Tenant SaaS Architecture** — One deployment serves multiple hospital groups, each with multiple branches, fully isolated data
- **Real-Time Operations** — Live queue boards, instant status updates, push notifications via Supabase Realtime
- **WhatsApp-First Patient Experience** — OTP login, appointment booking, queue notifications, and prescription delivery via WhatsApp Cloud API
- **n8n Workflow Backbone** — 40+ automated workflows handling scheduling, payments, reminders, and inter-system orchestration
- **9-Role RBAC System** — Granular access control from Platform Super Admin down to individual Lab Technicians

- **Zero-Config Deployment** — Dockerized standalone Next.js with auto-configured Supabase and n8n integration

Scale & Complexity

Metric	Count
TypeScript/TSX Files	150+
React Pages	49
API Routes	10
Custom Components	60+
Custom Hooks	16
Database Tables	25+
n8n Workflows	40+
User Roles	9
Authentication Methods	4 (PIN, Password, OTP, Supabase Auth)
Lines of Code (estimated)	~25,000

---

2. System Architecture

High-Level Architecture

```
                    +------------------+
                    |   Cloudflare     |
                    |   (DNS + SSL)    |
                    +--------+---------+
                             |
            +--------------+--------------+
            |                             |
     +-------+-------+            +--------+--------+
     |  Next.js App  |            |   n8n Instance   |
     |  (Standalone) |            |  (Docker/VPS)    |
     |  Port 3000    |            |  ainewworld.in   |
     +-------+-------+            +--------+--------+
             |                             |
             |    +-----------+            |
             +--->| Supabase  |<-----------+
                  | PostgreSQL|
                  | + Realtime|
                  | + Auth    |
                  +-----------+
                        |
            +---------+---------+
            |                   |
     +-------+------+   +-------+------+
```

```
            | WhatsApp     |   | Razorpay     |
            | Cloud API    |   | Payments     |
            | (Meta Graph) |   | (Test Mode)  |
            +--------------+   +--------------+
```

Request Flow

```
Browser → Next.js Middleware (auth + role check)
       → Page Component (client-side rendering)
       → Custom Hook (SWR + Supabase Realtime)
       → Supabase REST API (with RLS JWT)
       → PostgreSQL (tenant-isolated data)

For Operations:
Browser → Next.js API Route (session validation)
       → n8n Webhook (business logic execution)
       → Supabase (data persistence)
       → WhatsApp/Razorpay (external notifications)
       → Response → Client
```

Architecture Decisions

Decision	Choice	Rationale
Rendering	Client-Side (SPA-like)	Real-time dashboards need live data; SSR adds complexity for auth
Database	Supabase (PostgreSQL)	Realtime subscriptions, RLS, REST API, free tier for development
Auth	NextAuth v5 (JWT)	Multi-provider support, JWT for stateless sessions, Supabase JWT bridge
Styling	Tailwind CSS v4 + shadcn/ui	Rapid development, consistent design system, dark mode
Automation	n8n (self-hosted)	Visual workflow builder, 1000+ integrations, no vendor lock-in
State	SWR + Supabase Realtime	SWR for caching/revalidation, Realtime for push updates
Deployment	Docker Standalone	Minimal footprint, easy horizontal scaling

---

3. Technology Stack

Frontend

Technology	Version	Purpose
Next.js	15.x	React framework with App Router
React	19.x	UI library
TypeScript	5.x	Type safety

Tailwind CSS	4.x	Utility-first styling
shadcn/ui	Latest	Component library (Radix UI + Tailwind)
Framer Motion	Latest	Page transitions, micro-animations
Recharts	Latest	Analytics charts (lazy-loaded)
SWR	Latest	Data fetching with caching
Lucide React	Latest	575+ icons
Sonner	Latest	Toast notifications
@dnd-kit	Latest	Drag-and-drop (Kanban boards)
cmdk	Latest	Command palette / search
react-day-picker	Latest	Calendar widget
react-to-print	Latest	Print documents
date-fns + date-fns-tz	Latest	Date formatting (IST timezone)

Backend

Technology	Version	Purpose
Next.js API Routes	15.x	Server-side endpoints
NextAuth.js	5.x (beta)	Authentication (JWT strategy)
Supabase JS	Latest	Database client + Realtime + Auth
Jose	Latest	JWT signing (HS256)
Zod	Latest	Schema validation
pg	Latest	Direct PostgreSQL driver

Infrastructure

Technology	Purpose
Supabase (hosted)	PostgreSQL 15 + Realtime + Auth + Storage
n8n (self-hosted)	Workflow automation, 40+ workflows
Docker + Docker Compose	Containerized deployment
Cloudflare	DNS, SSL termination, DDoS protection
Hostinger VPS	n8n hosting (KVM 2, Mumbai datacenter)
WhatsApp Cloud API	Patient notifications via Meta Graph API
Razorpay	Payment processing (test mode)

Development Tools

Tool	Purpose
ESLint + Next.js config	Code linting
TypeScript strict mode	Type checking
shadcn CLI	Component scaffolding
Prettier (via ESLint)	Code formatting

---

4. Authentication & Security

Authentication Methods

AI-HOS supports **4 distinct authentication methods** serving different user types:

Method 1: PIN-Based Login (Hospital Staff)

```
Login Page → Select Role → Select Branch → Enter PIN
                                                ↓
                                        Validate against:
                                        - Branch admin_pin / reception_pin
                                        - Individual staff.pin
                                        - Doctor PIN (doctor table)
                                                ↓
                                        Build SessionUser → JWT → Dashboard
```

Supported Roles: Super Admin, Client Admin, Branch Admin, Admin, Doctor, Reception, Lab Tech, Pharmacist

Security Controls:

- Rate limiting: 5 failed attempts per 15-minute window
- PIN stored as plaintext in database (adequate for hospital environments with physical access control)
- Branch must have `status = 'active'` to allow login
- Rate limit key includes tenant + role + identifier for granular tracking

Method 2: Email/Password Login (Staff with Credentials)

```
Login Page → Email & Password Tab → Verify via Supabase Auth
                                                ↓
                                        Map email → user_credentials table
                                        Resolve: role, entity_table, entity_id
                                                ↓
                                        Build SessionUser → JWT → Dashboard
```

Security Controls:

- Passwords hashed by Supabase Auth (bcrypt)
- user_credentials table maps email to app identity
- Supabase Auth session immediately closed after verification (only used for password check)

Method 3: OTP-Based Login (Patients)

```
Patient Login → Enter Phone → Receive WhatsApp OTP
                                    ↓
                              Enter 6-digit OTP
                                    ↓
                              Verify: not expired, not used,
                              < 3 attempts, atomic consumption
                                    ↓
                              Build SessionUser → JWT → Patient Portal
```

Security Controls:

- 6-digit cryptographic OTP (`crypto.randomInt`)
- 5-minute expiration
- Maximum 3 OTP sends per phone per 15 minutes
- Maximum 5 verification attempts per phone per 15 minutes
- Atomic OTP consumption prevents concurrent double-use
- Generic error message prevents user enumeration
- Only first name returned (not full name)

Method 4: Supabase RLS JWT (Database Access)

```
NextAuth JWT callback → Sign custom Supabase JWT
                              ↓
                        Claims: tenant_id, user_role, sub
                              ↓
                        Browser client uses JWT as Authorization header
                              ↓
                        PostgreSQL RLS policies enforce tenant isolation
```

Session Architecture

```
SessionUser {
  id: string              // "admin-T001", "DOC001", "patient-919876543210"
  name: string            // Display name
  role: UserRole          // 9 possible roles
  tenantId: string        // Branch identifier
  hospitalName: string    // Branch display name
  clientId?: string       // Parent organization (multi-hospital groups)
  clientName?: string     // Organization name
  doctorId?: string       // Doctor-specific (DOCTOR role only)
  specialty?: string      // Doctor's specialty
  email?: string          // For password-auth users
  patientPhone?: string   // Patient identifier (PATIENT role only)
}
```

JWT Lifetime: 12 hours **Cookie:** HttpOnly, SameSite=Lax, Secure (production)

Security Headers

Applied globally via `next.config.ts`:

Header	Value	Purpose
X-Frame-Options	DENY	Prevent clickjacking
X-Content-Type-Options	nosniff	Prevent MIME sniffing
Strict-Transport-Security	max-age=63072000; includeSubDomains; preload	Force HTTPS
Referrer-Policy	strict-origin-when-cross-origin	Control referrer leakage
Permissions-Policy	camera=(), microphone=(), geolocation=()	Restrict browser APIs
X-DNS-Prefetch-Control	on	Performance optimization

Input Sanitization

- **PostgREST `.or()` injection prevention:** All user search input sanitized with `/[^a-zA-Z0-9\s\-\.]/g` before use in Supabase filter expressions
- **API parameter whitelisting:** Booking API enforces `actionSchemas` — only known fields per action are passed to n8n
- **Phone normalization:** All phone inputs stripped to digits, validated for 10/12-digit format
- **URL validation:** Queue notification URLs validated via `URL` hostname comparison against allowed origins

---

5. Multi-Tenancy Architecture

Entity Hierarchy

```
Platform (AI-HOS SaaS)
  └── Client (Hospital Group)        e.g., "Care Healthcare Pvt Ltd"
       └── Tenant/Branch (Hospital)  e.g., "Care Hospital - Hyderabad"
            ├── Doctors
            ├── Staff (Reception, Lab, Pharmacy)
            ├── Patients
            ├── Appointments
            ├── Queue Entries
            ├── Prescriptions
            ├── Lab Orders
            ├── Pharmacy Orders
            ├── Invoices
            ├── Admissions (IPD)
            └── Notifications
```

Data Isolation Layers

Layer 1: Middleware Route Protection

- Routes gated by role → users physically cannot navigate to unauthorized pages
- Example: A PHARMACIST role cannot access `/admin/*` or `/doctor/*` routes

Layer 2: Application-Level Filtering

- Every Supabase query includes `.eq("tenant_id", user.tenantId)`

- API routes inject `tenant_id` from session, ignoring client-supplied values
- Super Admin / Client Admin can switch branches explicitly

Layer 3: Supabase RLS Policies

- PostgreSQL Row-Level Security evaluates JWT `tenant_id` claim
- Even if application code has a bug, database rejects cross-tenant queries
- Service role key (server-side only) bypasses RLS for system operations

Layer 4: Session Enforcement

- Session JWT embeds `tenantId` at login time
- Cannot be modified client-side (signed with NEXTAUTH_SECRET)
- 12-hour expiry forces re-authentication

Tenant Resolution Flow

```
Staff Login:     User selects branch → tenant_id from branch selection
Client Admin:    Auto-selects first active branch → can switch via UI
Patient Login:   tenant_id from patients.tenant_id (registered hospital)
API Requests:    tenant_id injected from session (never trust client input)
Public Queue:    tenant_id from URL path parameter (/queue/[tenantId])
```

---

6. User Roles & Access Control

Role Matrix

Capability	Super Admin	Client Admin	Branch Admin	Admin	Doctor	Reception	Lab Tech	Pharmacist
Platform Management	Y	-	-	-	-	-	-	-
Cross-Client Analytics	Y	-	-	-	-	-	-	-
Client Management	Y	Y	-	-	-	-	-	-
Branch Settings	Y	Y	Y	Y	-	-	-	-
Staff Management	Y	Y	Y	Y	-	-	-	-
Doctor Management	Y	Y	Y	Y	-	-	-	-
Billing & Invoices	Y	Y	Y	Y	-	-	-	-
Analytics Dashboard	Y	Y	Y	Y	-	-	-	-

Patient Queue Management	Y	Y	Y	Y	-	Y	-	-
Appointment Booking	Y	Y	Y	Y	-	Y	-	-
Patient Registration	Y	Y	Y	Y	-	Y	-	-
IPD Admissions	Y	Y	Y	Y	-	Y	-	-
Consultation & Prescribe	-	-	-	-	Y	-	-	-
Doctor Schedule Mgmt	-	-	-	-	Y	-	-	-
Lab Order Processing	-	-	-	-	-	-	Y	-
Pharmacy Dispensing	-	-	-	-	-	-	-	Y
Self-Service Booking	-	-	-	-	-	-	-	-
View Own Records	-	-	-	-	-	-	-	-
Cancel Appointments	-	-	-	-	-	-	-	-

Route Protection (Middleware)

```
Public Routes (No Auth):
  /login, /patient-login, /forgot-password, /reset-password
  /queue/*, /api/*, /_next/*, /unauthorized

Protected Routes:
  /platform/*  → SUPER_ADMIN only
  /admin/*     → SUPER_ADMIN, CLIENT_ADMIN, BRANCH_ADMIN, ADMIN
  /reception/* → SUPER_ADMIN, CLIENT_ADMIN, BRANCH_ADMIN, ADMIN, RECEPTION
  /doctor/*    → SUPER_ADMIN, CLIENT_ADMIN, BRANCH_ADMIN, ADMIN, DOCTOR
  /pharmacy/*  → SUPER_ADMIN, CLIENT_ADMIN, BRANCH_ADMIN, ADMIN, PHARMACIST
  /lab/*       → SUPER_ADMIN, CLIENT_ADMIN, BRANCH_ADMIN, ADMIN, LAB_TECH
  /patient/*   → PATIENT only
```

---

7. Feature Modules

7.1 Reception Module

Purpose: Central hub for front-desk operations — patient check-in, walk-in booking, queue management, and IPD admissions.

Pages:

Page	Path	Features
Dashboard	`/reception`	Live queue board, real-time stats, quick actions, pending arrivals
Patients	`/reception/patients`	Patient search, registration, history
Appointments	`/reception/appointments`	Today's appointments, check-in management
Booking	`/reception/book`	Walk-in appointment booking form
Admissions	`/reception/admissions`	IPD admission management

Key Components:

- **Queue Board** — Real-time Supabase subscription, color-coded by status (waiting/in-consultation/completed), priority badges (emergency/urgent)
- **Quick Actions Panel** — Patient search → Check-in / Mark no-show / Admit
- **Booking Form** — 4-step wizard: Patient lookup → Doctor selection → Slot selection → Confirmation
- **Check-in Dialog** — Queue number assignment, WhatsApp notification, estimated wait time
- **Realtime Stats Banner** — Live counters: waiting, in-consultation, completed, avg wait time

7.2 Doctor Module

Purpose: Clinical workspace for consultations, prescriptions, patient history, and schedule management.

Pages:

Page	Path	Features
Dashboard	`/doctor`	Live queue, next patient, consultation timer, daily stats
Consultation	`/doctor/consult`	Active consultation with vitals, symptoms, diagnosis, prescribe
My Patients	`/doctor/patients`	Patient history, previous visits, prescriptions
Prescriptions	`/doctor/prescriptions`	Prescription history with print/export
Schedule	`/doctor/schedule`	Weekly schedule editor, date overrides, leave management

Key Features:

- **Next Patient Hero Card** — Prominent card showing next waiting patient with elapsed timer and "Start Consultation" button
- **Break Toggle** — Doctor can pause/resume queue (affects queue visibility)
- **Consultation Flow:** Start → Record vitals → Add symptoms/diagnosis → Prescribe medicines → Order lab tests → Complete
- **Auto-Creation:** Completing consultation creates pharmacy order + lab order + invoice automatically
- **Prescription Templates** — Save and reuse common prescriptions
- **Medicine Combobox** — Autocomplete from medicines database

7.3 Pharmacy Module

Purpose: Medicine dispensing workflow with Kanban board tracking.

Pages:

Page	Path	Features
Orders	`/pharmacy`	Kanban board: Pending → Preparing → Ready → Dispensed
Inventory	`/pharmacy/inventory`	Medicine stock management

Workflow:

```
Doctor Prescribes → Pharmacy Order (Pending)
                        ↓
                  Pharmacist clicks "Start Preparing"
                        ↓
                  Status → Preparing (pharmacist assigned)
                        ↓
                  Pharmacist clicks "Mark Ready"
                        ↓
                  Status → Ready (notification to reception)
                        ↓
                  Patient collects → "Dispense"
                        ↓
                  Status → Dispensed
                  Auto-generates invoice (idempotent)
                  Notification to admin
```

7.4 Lab Module

Purpose: Lab test order processing with sample tracking and results entry.

Pages:

Page	Path	Features
Orders	`/lab`	Dual view: Kanban board OR Table view

Workflow:

```
Doctor Orders Tests → Lab Order (Ordered)
                          ↓
                    Lab Tech → "Collect Sample"
                          ↓
                    Status → Sample Collected
                          ↓
                    Lab Tech → "Enter Results" (per-test values)
                          ↓
                    Status → Completed
                    Auto-generates invoice (idempotent)
                    Notification to doctor
                    Lab report available for print
```

Dual View Modes:

- **Kanban:** 4 columns with drag-drop cards
- **Table:** Full-featured table with search, filter, sort, and CSV export

7.5 Admin Module

Purpose: Hospital administration, staff management, billing, analytics, and system configuration.

Pages:

Page	Path	Features
Analytics	`/admin`	Revenue charts, doctor performance, appointment trends
Doctors	`/admin/doctors`	Doctor CRUD, specialty management
Patients	`/admin/patients`	Patient registry, visit history
Staff	`/admin/staff`	Staff CRUD (Reception, Lab, Pharmacy, Branch Admin)
Billing	`/admin/billing`	Invoice management, payment tracking, revenue analytics
Settings	`/admin/settings`	Branch name, PINs, consultation fee, ward configuration

Analytics Features:

- Date range filter with presets (today, yesterday, 7 days, 30 days)
- Line chart: Daily appointment trends
- Pie chart: Status breakdown
- Bar chart: Revenue trends
- Doctor performance cards (consultations, avg time, revenue)
- Cross-branch analytics (for Client Admin role)
- CSV export of raw data
- Animated counters with sparklines

7.6 Patient Portal

Purpose: Self-service patient experience — booking, records, prescriptions, billing.

Pages:

Page	Path	Features
Dashboard	`/patient`	Welcome banner, stats, upcoming appointments, recent prescriptions
Book	`/patient/book`	5-step booking wizard
Appointments	`/patient/appointments`	All/Upcoming/Past with cancel capability
Prescriptions	`/patient/prescriptions`	Medicine history with details
Lab Results	`/patient/lab`	Test results with normal range indicators
Invoices	`/patient/invoices`	Bills with payment status

OP Pass	`/patient/op-pass`	Digital outpatient pass with QR code
Profile	`/patient/profile`	Personal information management

Booking Flow:

```
Select Hospital → Select Specialty → Select Doctor
        ↓               ↓                ↓
  (from previous   (from n8n     (filtered by
   appointments)   webhook)       specialty)
        ↓
  Select Date (7-day view) → Select Time Slot
        ↓                        ↓
    (from n8n              (from doctor_schedules
     availability)          + date_overrides)
        ↓
  Confirm Booking → Payment Link (Razorpay)
        ↓
  Booking confirmed via n8n → WhatsApp notification
```

7.7 IPD (Inpatient Department)

Purpose: Inpatient management — admission, ward tracking, daily charges, nursing notes, transfer, and discharge.

Components:

Component	Features
Bed Map	Visual ward layout with occupancy status
Admission Detail	Patient info, doctor assignment, admission history
Daily Charges	Track room, medicines, procedures, misc charges
Nursing Notes	Vitals recording, observations, medication administration
Transfer Dialog	Ward/bed transfer with history tracking
Discharge Dialog	Billing preview, discharge summary, invoice generation

7.8 Platform Module (SaaS Super Admin)

Purpose: Platform-level management for the SaaS operator.

Pages:

Page	Path	Features
Overview	`/platform`	Cross-tenant metrics
Analytics	`/platform/analytics`	Aggregate analytics across all clients
Clients	`/platform/clients`	Client CRUD, subscription management
Branches	`/platform/branches`	All branches across all clients

Doctors	`/platform/doctors`	All doctors across platform
Patients	`/platform/patients`	All patients across platform
Health	`/platform/health`	System health monitoring
Logs	`/platform/logs`	Audit trail viewer
Plans	`/platform/plans`	Subscription plan management
Settings	`/platform/settings`	Platform configuration

7.9 Public Queue Display

Purpose: Kiosk/TV display for hospital waiting rooms.

Path: `/queue/[tenantId]` (no authentication required)

Features:

- Full-screen queue board
- Live clock (IST timezone, updates every second)
- Real-time patient queue (Supabase subscription + 10s polling)
- Audio alert (Web Audio API beep) when patient called
- Visual flash effect on status change
- Color-coded: waiting (default) vs in-consultation (highlighted)
- Designed for wall-mounted landscape displays

---

8. User Flows & Journeys

Journey 1: New Patient Walk-In (Complete OPD Flow)

```
Patient arrives at hospital
        ↓
RECEPTION: Search patient by phone
        ↓
[Not Found] → Register new patient (name, phone, age, gender)
        ↓
RECEPTION: Book walk-in appointment
  → Select doctor/specialty → Select available slot → Confirm
        ↓
System: Creates appointment (status: confirmed)
        ↓
RECEPTION: Check-in patient
  → Assigns queue number → Creates queue entry (status: waiting)
  → Sends WhatsApp notification with queue number & estimated wait
        ↓
Patient waits in lobby (monitors public queue display)
        ↓
DOCTOR: Sees patient in queue → Clicks "Start Consultation"
  → Queue status → in_consultation
  → Public display updates in real-time
        ↓
```

```
DOCTOR: Records vitals, symptoms, diagnosis
  → Prescribes medicines (→ creates pharmacy_order)
  → Orders lab tests (→ creates lab_order)
  → Completes consultation
        ↓
Queue status → completed | Appointment status → completed
Auto-creates consultation invoice
        ↓
PHARMACY: Sees pending order → Prepares → Marks ready → Dispenses
  → Auto-creates pharmacy invoice
        ↓
LAB: Sees pending order → Collects sample → Enters results
  → Auto-creates lab invoice → Notifies doctor
        ↓
Patient receives WhatsApp: prescription details + follow-up date
Patient can view everything in Patient Portal
```

Journey 2: Patient Self-Service Online Booking

```
Patient visits /patient-login
        ↓
Enter phone number → Receive OTP on WhatsApp → Verify OTP
        ↓
Patient Dashboard: View upcoming appointments, prescriptions, bills
        ↓
Click "Book Appointment"
        ↓
Select Specialty → Select Doctor → Select Date → Select Time Slot
        ↓
Confirm → n8n creates appointment → Returns payment link
        ↓
Pay via Razorpay → Razorpay webhook confirms payment
        ↓
n8n: Creates OP Pass (15-day validity) → Sends reminders
        ↓
Appointment day: Check-in at reception or auto-queue
```

Journey 3: IPD (Inpatient) Journey

```
RECEPTION: Admit patient via Admit Dialog
  → Select ward → Select bed → Assign doctor
  → Creates admission record (status: admitted)
        ↓
NURSING: Add daily nursing notes
  → Record vitals (temp, BP, pulse, SpO2)
  → Log medications administered
  → Add observations
        ↓
DOCTOR: Review admission → Order treatments
  → Prescribe medicines → Order lab tests
        ↓
```

```
ADMIN: Add daily charges (room, procedures, medicines)
        ↓
[If needed] Transfer patient to different ward/bed
  → Transfer history maintained
        ↓
DOCTOR: Approve discharge
        ↓
RECEPTION: Process discharge
  → Billing preview (all charges aggregated)
  → Generate discharge summary
  → Create final invoice
  → Update bed status → Free
        ↓
Print discharge summary + final invoice
```

Journey 4: Doctor's Day

```
DOCTOR: Login with PIN → Arrives at Doctor Dashboard
        ↓
View daily stats: patients today, waiting, completed, avg consult time
        ↓
[Optional] Set break status (pauses queue)
        ↓
See "Next Patient" hero card with:
  - Patient name, phone, priority badge
  - Time waiting (elapsed timer)
  - Recent visit history (last 3)
        ↓
Click "Start Consultation" → Redirected to /doctor/consult
        ↓
Consultation workspace:
  ├─ Patient info panel (demographics, allergies, conditions)
  ├─ Vitals entry (temp, BP, pulse, weight, SpO2)
  ├─ Symptoms (text)
  ├─ Diagnosis (text)
  ├─ Prescriptions (medicine combobox with dosage/frequency/duration)
  ├─ Lab test orders (multi-select from available tests)
  └─ Follow-up date picker
        ↓
Click "Complete Consultation"
  → Prescription saved → Pharmacy order created
  → Lab order created → Invoice generated
  → Queue status updated → Next patient loads
        ↓
View completed patients in dashboard
Manage schedule in /doctor/schedule
```

---

9. Database Schema

Entity Relationship Overview

```
clients (1) ──────< tenants (1) ──────< doctors
                        │                  │
                        ├──────< staff     │
                        │                  │
                        ├──────< patients  │
                        │         │        │
                        │         ├───< appointments ───< queue_entries
                        │         │        │
                        │         │        ├───< prescriptions ───< pharmacy_orders
                        │         │        │
                        │         │        └───< lab_orders
                        │         │
                        │         ├───< admissions
                        │         │
                        │         └───< invoices
                        │
                        ├──────< doctor_schedules
                        ├──────< date_overrides
                        ├──────< slot_locks
                        ├──────< notifications
                        └──────< audit_logs
```

Core Tables

Table	Primary Key	Key Columns	Notes
`clients`	client_id	name, admin_pin, status, plan, max_branches	Hospital groups
`tenants`	tenant_id	hospital_name, client_id, admin_pin, reception_pin, status	Individual branches
`platform_admins`	admin_id	email, pin, name	Super admins
`user_credentials`	email	role, entity_table, entity_id, tenant_id, client_id	Password auth mapping
`doctors`	doctor_id	tenant_id, name, specialty, pin, consultation_fee, status	Doctors with schedule
`staff`	staff_id	tenant_id, name, role, pin, phone, status	Non-doctor staff
`patients`	phone	tenant_id, name, age, gender, email, visit_count	Patient registry
`dependents`	dependent_id	patient_phone, name, age, gender, relation	Family members
`appointments`	booking_id	tenant_id, doctor_id, patient_phone, date, time, status	Bookings
`queue_entries`	queue_id	tenant_id, doctor_id, booking_id, queue_number, status, priority	Live queue
`prescriptions`	prescription_id	booking_id, patient_phone, doctor_id, items (JSONB), diagnosis	Rx records

`pharmacy_orders`	order_id	tenant_id, prescription_id, patient_phone, items (JSONB), status	Medicine orders
`lab_orders`	order_id	tenant_id, patient_phone, doctor_id, tests (JSONB), status	Lab test orders
`invoices`	invoice_id	tenant_id, patient_phone, type, items (JSONB), total, payment_status	Bills
`admissions`	admission_id	tenant_id, patient_phone, doctor_id, ward, bed, status	IPD records
`op_passes`	op_pass_id	booking_id, patient_phone, valid_from, valid_to, reschedules_remaining	OP passes
`doctor_schedules`	id	doctor_id, tenant_id, day_of_week, start_time, end_time, slot_duration	Weekly schedule
`date_overrides`	id	doctor_id, tenant_id, date, leave, custom_start, custom_end	Date exceptions
`slot_locks`	id	doctor_id, date, time, locked_until	Prevent double-booking
`notifications`	id	tenant_id, type, title, message, target_role, read	System alerts
`patient_otps`	id	phone, otp, verified, expires_at, attempts	OTP records
`audit_logs`	id	tenant_id, action, actor, details	Audit trail

Key Constraints

```sql
-- Prevent double-booking
CREATE UNIQUE INDEX idx_no_double_booking
  ON appointments(doctor_id, date, time)
  WHERE status = 'confirmed';

-- Prevent duplicate queue numbers
CREATE UNIQUE INDEX uq_queue_number
  ON queue_entries(tenant_id, date, queue_number);

-- Prevent duplicate slot locks (race condition guard)
CREATE UNIQUE INDEX uq_slot_lock
  ON slot_locks(doctor_id, date, time);
```

---

10. API Architecture

API Routes

Route	Method	Auth	Purpose
`/api/auth/[...nextauth]`	GET/POST	Public	NextAuth session management

`/api/auth/tenants`	GET	Public	List clients & branches for login page
`/api/auth/create-password`	POST	Authenticated	Create email/password credentials
`/api/auth/forgot-password`	POST	Public	Password reset request
`/api/booking`	POST	Staff (6 roles)	Proxy to n8n webhooks with param sanitization
`/api/patient-booking`	POST	PATIENT	Patient self-service booking
`/api/patient-appointment`	POST	PATIENT	Cancel patient appointments
`/api/patient-auth/send-otp`	POST	Public	Generate and send OTP
`/api/queue/notify`	POST	Authenticated	Send WhatsApp queue notification
`/api/platform`	POST	SUPER_ADMIN	Platform management operations

API Security Layers

```
Request → Rate Limiting (in-memory sliding window)
       → Session Validation (auth() from NextAuth)
       → Role Authorization (allowedRoles check)
       → Tenant Injection (session.tenantId, never trust client)
       → Parameter Sanitization (whitelist per action)
       → Webhook Timeout (15s AbortSignal)
       → Response Validation (check response.ok)
       → Error Handling (502 for n8n failures, 500 for app errors)
```

Booking API Action Schema

```
const actionSchemas = {
  "check-availability": ["tenant_id", "doctor_id", "date", "specialty"],
  "book-appointment":   ["tenant_id", "patient_phone", "patient_name", "patient_type",
                          "doctor_id", "doctor_name", "specialty", "date", "time",
                          "source", "booked_by_whatsapp_number"],
  "cancel-appointment": ["tenant_id", "booking_id", "patient_phone"],
  "list-specialties":   ["tenant_id"],
  "patient-lookup":     ["tenant_id", "phone"],
  "save-patient":       ["tenant_id", "phone", "name", "age", "gender", "email", "address"],
}
```

Any field not in the schema is silently dropped before forwarding to n8n.

---

11. Real-Time Engine

Supabase Realtime Subscriptions

AI-HOS uses Supabase's PostgreSQL LISTEN/NOTIFY for real-time updates:

Table	Subscribers	Events	Purpose
`queue_entries`	Reception, Doctor, Public Queue	INSERT, UPDATE, DELETE	Live queue updates
`notifications`	All dashboard users	INSERT	Real-time alerts
`appointments`	Reception, Doctor	UPDATE	Status change notifications
`pharmacy_orders`	Pharmacy	INSERT, UPDATE	New orders, status changes
`lab_orders`	Lab	INSERT, UPDATE	New orders, status changes
`admissions`	IPD	INSERT, UPDATE	Admission status changes

Real-Time Architecture

```
Supabase PostgreSQL
  │
  ├─ LISTEN/NOTIFY channel per table
  │
  ├─ Supabase Realtime Server (WebSocket)
  │
  └─ Browser Client (supabase.channel().on('postgres_changes'))
       │
       ├─ useQueue() hook → merges updates into local state
       ├─ useNotifications() hook → shows toast + updates badge
       └─ SWR mutate() → triggers re-fetch for complex queries
```

Fallback Polling

For reliability, critical features also poll at fixed intervals:

- Queue entries: 10-second polling + realtime
- Public queue display: 10-second polling
- Notifications: Realtime only (with reconnection)

---

12. Workflow Automation (n8n)

n8n Instance

Property	Value
URL	https://ainewworld.in
Version	2.35.2
Mode	Queue (workers + Redis)
Database	PostgreSQL 15

Hosting	Hostinger VPS (KVM 2, Mumbai)
Active Workflows	40+

Workflow Categories

Patient & Booking Workflows (13)

Workflow	Webhook	Purpose
WhatsApp Cloud API	`/whatsapp-cloud`	Conversational AI bot (10 OpenAI tools)
Patient Lookup	`/patient-lookup`	Search patient by phone
Save Patient	`/save-patient`	Create/update patient record
Save Dependent	`/save-dependent`	Add family member
List Specialties	`/list-specialties`	Get doctors grouped by specialty
Availability 7 Days	`/cal-availability`	Calculate available slots
Book Appointment	`/book-appointment`	Create booking + payment link
Cancel Appointment	`/cancel-appointment`	Cancel with refund
Reschedule Appointment	`/reschedule-appointment`	Change date/doctor
List Appointments	`/list-appointments`	Patient appointment history
Check OP Pass	`/check-op-pass`	Verify outpatient pass
Verify Appointment	`/verify-appointment`	Validate booking exists
Send Patient OTP	`/send-patient-otp`	WhatsApp OTP delivery

Payment Workflows (3)

Workflow	Purpose
Payment Link Generator	Create Razorpay payment links
Payment Confirmation	Process Razorpay webhook callbacks
Send Payment Emails	Email receipts post-payment

Scheduled Workflows (5)

Workflow	Schedule	Purpose
Appointment Reminders	Every 15 min	WhatsApp reminders for upcoming appointments
Slot Lock Cleanup	Every 2 min	Release expired slot locks
OP Pass Expiry Checker	Daily midnight	Expire outdated OP passes
Cleanup Expired Unpaid	Every hour	Cancel unpaid expired bookings

Booking Events Processor	Every 1 min	Process queued booking events

Dashboard & Admin Workflows (5)

Workflow	Purpose
Doctor Login	PIN validation for doctor dashboard
Doctor Appointments	Fetch doctor's daily appointments
Doctor Prescribe	Save prescriptions from doctor dashboard
Doctor Leave	Manage doctor leave days
Doctor Dashboard	Dashboard statistics

Admin Workflows (3)

Workflow	Purpose
Admin API	12 admin actions (CRUD operations)
Hospital Admin Dashboard	HTML admin portal
Reception Dashboard	HTML reception portal

System Reliability (8)

Workflow	Schedule	Purpose
Admin Error Alerts	Error trigger	WhatsApp alerts to admin on any workflow failure
System Health Monitor	Every 15 min	Endpoint health checks
Automated Testing Pipeline	Every hour	Integration test suite
Engineering Report Generator	Hourly at :30	Performance reports
Bug Detection & Self-Healing	Every 10 min	Anomaly detection
Performance Optimization Monitor	Every 30 min	Response time tracking
Security Audit Engine	Every 6 hours	Security scanning
Feature Discovery Engine	Daily 8 AM	Feature gap analysis

WhatsApp Bot Architecture

The WhatsApp bot uses OpenAI with 10 tool functions:

1. `lookup_patient` — Search by phone
2. `save_patient` — Register new patient
3. `save_dependent` — Add family member
4. `list_specialties` — Show available specialties
5. `check_availability_7days` — Show available slots
6. `book_appointment` — Create booking

7. `cancel_appointment` — Cancel booking
8. `list_appointments` — Show patient's bookings
9. `check_op_pass` — Verify OP pass
10. `reschedule_appointment` — Change appointment

Button Markers: The AI generates structured markers like `[BUTTONS:specialty:Cardiology,Orthopedics]` that the workflow converts to WhatsApp interactive buttons/lists.

---

13. Design System & UX

Design Philosophy

AI-HOS follows an **Apple Health-inspired** design system with:

- Clean, spacious layouts with generous whitespace
- Soft gradients and glass morphism effects
- Micro-animations for state changes
- Dark mode support throughout
- Mobile-first responsive design

Color System

Color	Light	Dark	Usage
Primary	#007AFF	#0A84FF	Actions, links, active states
Background	#F2F2F7	#000000	Page background
Card	#FFFFFF	#1C1C1E	Card surfaces
Destructive	#FF3B30	#FF453A	Errors, cancel actions
Success	#34C759	#30D158	Confirmed, completed
Warning	#FF9500	#FF9F0A	Pending, attention

Typography

- **Primary Font:** Plus Jakarta Sans (400, 500, 600, 700)
- **Code Font:** JetBrains Mono
- **Scale:** text-xs (10px) → text-2xl (24px)
- **Heading hierarchy:** Bold weights with text-foreground

Glass Morphism Effects

```
.glass {
  background: rgba(255, 255, 255, 0.72);
  backdrop-filter: blur(20px);
  border: 1px solid rgba(0, 0, 0, 0.06);
}
```

Applied to: sidebar, cards, dialogs, dropdowns

Gradient System

```
.gradient-blue    { from: #007AFF → to: #00C6FF }
.gradient-green   { from: #34C759 → to: #30D158 }
.gradient-orange  { from: #FF9500 → to: #FF6B00 }
.gradient-purple  { from: #AF52DE → to: #5856D6 }
.gradient-red     { from: #FF3B30 → to: #FF6259 }
.gradient-teal    { from: #5AC8FA → to: #64D2FF }
```

Premium variants add a third color stop for depth.

Animation System

Animation	Duration	Usage
fade-in	400ms	Page entries, list items
slide-up	400ms	Cards, dialogs
scale-in	300ms	Buttons, badges
pulse-soft	2000ms	Active indicators
ping-dot	1500ms	Notification dots
shimmer	2000ms	Loading skeletons
pulse-glow	2000ms	Active consultation cards

Component Patterns

- **Stat Cards:** Icon + value (AnimatedCounter) + label, with gradient background
- **Kanban Boards:** Drag-drop columns with AnimatePresence for smooth transitions
- **Data Tables:** @tanstack/react-table with sorting, filtering, pagination
- **Dialogs:** Premium gradient headers with status pipeline indicators
- **Empty States:** Centered icon + message + action button
- **Search Bars:** Icon + input with debounce (300ms)
- **Badges:** Color-coded by status with consistent mapping

---

14. Print & Document Generation

Printable Documents

Document	Component	Format
Prescription	`prescription-print.tsx`	A4 portrait, hospital letterhead
Invoice / Receipt	`invoice-print.tsx`	A4 portrait, itemized bill
Lab Report	`lab-report-print.tsx`	A4 portrait, normal range indicators
Discharge Summary	`discharge-summary-print.tsx`	A4 portrait, full admission history

Analytics Report	`analytics-report-print.tsx`	A4 landscape, charts + tables
Queue Token	`print-token.ts`	90mm x 155mm card, QR code

Print Architecture

```
PrintButton component → react-to-print library → window.print()
                                ↓
                     PrintLayout wrapper (A4 dimensions)
                                ↓
                     Document component (content only, no nav/footer)
```

---

15. Deployment & Infrastructure

Docker Configuration

```
# Multi-stage build
FROM node:20-alpine AS builder
  → npm ci
  → npm run build (standalone output)

FROM node:20-alpine AS runner
  → Copy standalone build
  → EXPOSE 3000
  → CMD ["node", "server.js"]
```

Environment Variables

Variable	Required	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Yes	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Yes	Browser client key
`SUPABASE_SERVICE_ROLE_KEY`	Yes	Server operations (bypasses RLS)
`SUPABASE_JWT_SECRET`	Yes	Custom JWT signing
`NEXTAUTH_URL`	Yes	Canonical app URL
`NEXTAUTH_SECRET`	Yes	Session encryption
`N8N_WEBHOOK_URL`	Yes	n8n base URL
`WA_PHONE_NUMBER_ID`	Yes	WhatsApp sender
`WA_ACCESS_TOKEN`	Yes	WhatsApp API token
`NEXT_PUBLIC_APP_URL`	Yes	Public app URL
`DEFAULT_TENANT_ID`	Optional	Default branch

Infrastructure Diagram

```
┌──────────────────────────────────────────┐
│                Cloudflare                 │
│  DNS: ainewworld.in  │  SSL: Full (Strict)  │
│  DDoS Protection     │  Edge Caching        │
└──────────┬──────────────────────────┬──────┘
           │                          │
┌──────────▼──────────┐  ┌──────────▼──────────┐
│   Hostinger VPS     │  │   Vercel / Docker   │
│   72.61.238.6       │  │   (Next.js App)     │
│                     │  │                     │
│  ┌───────────────┐  │  │  Port 3000          │
│  │   Traefik     │  │  │  Standalone build   │
│  │   (Reverse    │  │  │                     │
│  │    Proxy)     │  │  └─────────────────────┘
│  └──────┬────────┘  │
│         │           │
│  ┌──────▼────────┐  │    ┌──────────────────┐
│  │    n8n        │  │    │    Supabase      │
│  │   Port 5678   │──┼──>│   (Hosted)       │
│  │   40+ flows   │  │    │   PostgreSQL 15  │
│  └───────────────┘  │    │   Realtime       │
│                     │    │   Auth           │
│  ┌───────────────┐  │    └──────────────────┘
│  │   Redis       │  │
│  │   (Queue)     │  │    ┌──────────────────┐
│  └───────────────┘  │    │   Meta Graph API │
│                     │    │   WhatsApp Cloud │
│  ┌───────────────┐  │    └──────────────────┘
│  │  PostgreSQL   │  │
│  │  (n8n DB)     │  │    ┌──────────────────┐
│  └───────────────┘  │    │   Razorpay       │
│                     │    │   (Payments)     │
└─────────────────────┘    └──────────────────┘
```

---

16. Security Audit Summary

Audit Results (March 2026)

A 10-phase production readiness audit was conducted covering architecture, functionality, security, performance, and end-to-end testing.

Issues Found & Fixed

Category	Critical	High	Medium	Low	Fixed
Security	1	3	3	2	9/9
Performance	1	2	2	0	5/5
Webhook Resilience	0	6	3	3	9/12

Data Integrity	2	3	2	0	5/7
Total	**4**	**14**	**10**	**5**	**28/33**

Key Fixes Applied

1. **PostgREST Injection Prevention** — Sanitized all `.or()` filter inputs across 5 files
2. **Security Headers** — HSTS, X-Frame-Options, CSP-adjacent headers on all routes
3. **Webhook Timeouts** — 15-second AbortSignal on all n8n webhook calls
4. **Response Validation** — Check `response.ok` before parsing n8n responses
5. **OTP Delivery Feedback** — Return 502 error (not silent success) when OTP fails
6. **Origin Validation** — URL hostname comparison instead of string prefix
7. **Query Limits** — `.limit()` caps on 12+ unbounded Supabase queries
8. **O(n^2) Elimination** — Map-based lookups in admin dashboard analytics
9. **Invoice Idempotency** — Order-ID-based invoice IDs prevent duplicates
10. **Queue Number Safety** — Insert-first-then-count prevents duplicate numbers
11. **OTP Atomic Consumption** — WHERE guard prevents concurrent double-use
12. **Consultation Error Granularity** — Specific error messages for partial saves

Remaining Manual Actions

Action	Location	Severity
Run RLS migration SQL	Supabase SQL Editor	Critical
Add queue_number UNIQUE constraint	Supabase SQL Editor	High
Add appointment double-booking index	Supabase SQL Editor	High
Rotate SUPABASE_SERVICE_ROLE_KEY	Supabase Dashboard	High
Rotate WA_ACCESS_TOKEN	Meta Business Suite	High
Set N8N_WEBHOOK_URL env var	Production environment	Medium

---

17. Performance Optimizations

Implemented Optimizations

Optimization	Impact	Location
Standalone build output	~70% smaller Docker image	next.config.ts
Static asset caching (1 year, immutable)	Instant cached asset loads	next.config.ts
Lazy-loaded charts (Recharts)	-40KB initial bundle	lazy-recharts.tsx
SWR client caching	Reduced API calls, instant UI	All hooks
Query limit caps	Prevents OOM on large datasets	Admin, staff, schedule pages
Map-based O(1) lookups	Eliminated O(n^2) in analytics	Admin dashboard
Debounced search (300ms)	Reduced query frequency	Search bars

AnimatePresence popLayout	Efficient Kanban animations	Pharmacy, lab Kanban
Skeleton loading states	Perceived performance boost	All pages
IST timezone caching	Avoids repeated Date operations	date utility

Recommended Future Optimizations

- Redis-based rate limiting (replace in-memory Map)
- Supabase connection pooling (PgBouncer)
- Edge caching for public queue pages
- Image optimization via next/image
- Bundle analysis and tree-shaking audit

---

18. Known Issues & Limitations

Current Limitations

Issue	Severity	Workaround
RLS policies not yet active	Critical	Run migration SQL in Supabase
In-memory rate limiting resets on restart	Medium	Acceptable for single-instance; use Redis for multi-instance
New patients cannot self-book (no tenant_id)	Medium	Must have prior appointment or reception check-in first
Phone format inconsistency (10 vs 12 digit)	Medium	Dual-format queries handle both; normalize in future migration
IPD module partially wired	Medium	Components exist but routes incomplete
Framer Motion not lazy-loaded	Low	~40KB gzip; consider dynamic import
No circuit breaker for n8n	Low	15s timeout provides basic protection
Chart data not memoized	Low	Minor re-render overhead

Browser Support

- Chrome 90+ (primary target)
- Firefox 90+
- Safari 15+
- Edge 90+
- Mobile: iOS Safari 15+, Chrome Android 90+

---

19. Complexity Analysis

Codebase Metrics

Metric	Value	Assessment

Total files	~150 TypeScript/TSX	Moderate complexity
Largest file	auth/options.ts (~735 lines)	Could be split but manageable
Average component size	~150 lines	Well-structured
Custom hooks	16	Good abstraction
Shared utilities	18 lib files	Clean separation
Type definitions	2 files, 30+ interfaces	Strong typing

Dependency Graph Complexity

```
High Coupling:
  auth/options.ts → supabase/server.ts → 5 database tables
  session-provider.tsx → supabase-auth-provider.tsx → supabase/client.ts

Medium Coupling:
  booking-form.tsx → 3 API routes → n8n webhooks
  check-in-dialog.tsx → queue hook → realtime subscriptions

Low Coupling:
  UI components (shadcn/ui) → independent, composable
  Print components → standalone, no side effects
  Utility functions → pure functions
```

Architectural Complexity Rating

Area	Complexity	Reason
Authentication	High	4 login methods, 9 roles, JWT bridge to Supabase RLS
Multi-Tenancy	High	3-level hierarchy (Platform → Client → Tenant), data isolation at 4 layers
Real-Time	Medium	Supabase subscriptions + SWR + polling fallback
API Layer	Medium	Parameter sanitization, tenant injection, n8n proxy
UI Components	Low-Medium	shadcn/ui base + domain-specific wrappers
Database	Medium	25+ tables, JSONB fields, multi-format phone handling
Workflow (n8n)	High	40+ workflows, WhatsApp bot with 10 tools, scheduled jobs
Payment	Low	Razorpay test mode with webhook callback

Lines of Code Estimate by Module

Module	Estimated LoC	% of Total
Authentication & Auth	~2,500	10%
Reception Module	~3,000	12%

Doctor Module	~3,500	14%
Pharmacy Module	~1,500	6%
Lab Module	~1,500	6%
Admin Module	~4,000	16%
Patient Portal	~3,000	12%
Platform (SaaS)	~2,000	8%
IPD Module	~1,500	6%
Shared (hooks, utils, UI)	~2,500	10%
Total	**~25,000**	**100%**

---

20. Future Roadmap

Phase 1: Production Launch Prerequisites

- [ ] Run RLS migration in Supabase
- [ ] Add database constraints (queue uniqueness, appointment double-booking)
- [ ] Rotate all credentials (Supabase key, WhatsApp token)
- [ ] Deploy Next.js app to production (Docker or Vercel)
- [ ] SSL certificate for app domain
- [ ] Production environment variables

Phase 2: Immediate Enhancements

- [ ] Complete IPD module routing and admission workflow
- [ ] Add circuit breaker pattern for n8n webhook calls
- [ ] Redis-based rate limiting for multi-instance deployment
- [ ] Phone number normalization migration (standardize to 12-digit)
- [ ] Email notifications (Gmail SMTP with App Password)
- [ ] Patient registration without prior appointment

Phase 3: Feature Expansion

- [ ] Multi-hospital WhatsApp routing (second phone number)
- [ ] Patient feedback/rating system
- [ ] Appointment rescheduling from patient portal
- [ ] Insurance claim management
- [ ] Medical records sharing (patient consent-based)
- [ ] Telemedicine / video consultation integration

Phase 4: Scale & Enterprise

- [ ] Horizontal scaling (multiple Next.js instances behind load balancer)
- [ ] CDN for static assets (Cloudflare Pages)
- [ ] Database read replicas for analytics

- [ ] Audit log analytics dashboard
- [ ] HIPAA compliance review
- [ ] Mobile app (React Native or Flutter)

Appendix A: File Structure Reference

```
ai-hos/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/page.tsx
│   │   │   ├── patient-login/page.tsx
│   │   │   ├── forgot-password/page.tsx
│   │   │   └── reset-password/page.tsx
│   │   ├── (dashboard)/
│   │   │   ├── admin/ (6 pages)
│   │   │   ├── doctor/ (5 pages)
│   │   │   ├── reception/ (5 pages)
│   │   │   ├── pharmacy/ (2 pages)
│   │   │   ├── lab/ (1 page)
│   │   │   └── layout.tsx
│   │   ├── (patient)/
│   │   │   ├── patient/ (8 pages)
│   │   │   └── layout.tsx
│   │   ├── (platform)/
│   │   │   ├── platform/ (10 pages)
│   │   │   └── layout.tsx
│   │   ├── queue/
│   │   │   ├── [tenantId]/page.tsx
│   │   │   └── [tenantId]/doctor/[doctorId]/page.tsx
│   │   ├── api/ (10 routes)
│   │   ├── layout.tsx (root)
│   │   └── globals.css
│   ├── components/
│   │   ├── ui/ (30+ base components)
│   │   ├── layout/ (7 files)
│   │   ├── reception/ (9 files)
│   │   ├── doctor/ (2 files)
│   │   ├── pharmacy/ (2 files)
│   │   ├── lab/ (1 file)
│   │   ├── ipd/ (6 files)
│   │   ├── admin/ (3 files)
│   │   ├── patient/ (1 file)
│   │   ├── print/ (7 files)
│   │   ├── shared/ (6 files)
│   │   ├── auth/ (1 file)
│   │   └── providers/ (2 files)
│   ├── hooks/ (16 custom hooks)
│   ├── lib/
│   │   ├── auth/ (5 files)
```

```
│   │   ├── supabase/ (3 files)
│   │   ├── utils/ (5 files)
│   │   ├── n8n/ (1 file)
│   │   ├── notifications.ts
│   │   ├── rate-limit.ts
│   │   ├── audit.ts
│   │   ├── print-token.ts
│   │   └── print-prescription.ts
│   └── types/ (2 files)
├── supabase/
│   └── migrations/ (SQL files)
├── public/ (static assets)
├── next.config.ts
├── package.json
├── tsconfig.json
├── Dockerfile
└── docker-compose.yml
```

Appendix B: Glossary

Term	Definition
Tenant	A hospital branch — the primary data isolation boundary
Client	A hospital organization/group that owns one or more tenants
OP Pass	Outpatient pass — 15-day validity card issued after payment
Queue Entry	A patient's position in the doctor's queue for the day
Booking ID	Unique appointment identifier (format: BK + timestamp)
Slot Lock	Temporary 5-minute reservation to prevent double-booking during payment
RLS	Row-Level Security — PostgreSQL feature for tenant data isolation
n8n	Open-source workflow automation platform (self-hosted)
SWR	Stale-While-Revalidate — React data fetching library
IST	Indian Standard Time (UTC+5:30) — all times stored and displayed in IST

Document generated on March 8, 2026 AI-HOS v1.0 — Advera Healthcare Technologies